

第 6 章-深度神经网络 – 第 6.3 节-深度网络

《深度学习基础》课程小组

2026 年 5 月 2 日

目录

1	6.3.1 Hierarchical representations	4
2	6.3.2 Distributed representations	5
3	6.3.3 Representation learning	6
4	6.3.4 Transfer learning	9
5	6.3.5 Contrastive learning	11
6	6.3.6 General network architectures	16
7	6.3.7 Tensors	17

We have motivated the development of neural networks by making the basis functions of a linear regression or classification model themselves be governed by learnable parameters, giving rise to the two-layer network model shown in Figure 6.9.

For many years, this was the most widely used architecture, primarily because it proved difficult to train networks with more than two layers effectively.

However, extending neural networks to have more than two layers, known as deep neural networks, brings many advantages as we will discuss shortly, and recent advances in techniques for training neural networks are effective for networks with many layers.

We can easily extend the two-layer network architecture (6.12) to any finite number L of layers, in which layer $\ell = 1, \dots, L$ computes the following function:

$$\mathbf{z}^{(\ell)} = h^{(\ell)}(\mathbf{W}^{(\ell)}\mathbf{z}^{(\ell-1)}) \quad (6.19)$$

where $h^{(\ell)}$ denotes the activation function associated with layer ℓ , and $\mathbf{W}^{(\ell)}$ denotes the corresponding matrix of weight and bias parameters.

Also, $\mathbf{z}^{(0)} = \mathbf{x}$ represents the input vector and $\mathbf{z}^{(L)} = \mathbf{y}$ represents the output vector.

输入层称为第 0 层。输出层称为第 L 层。

Note that there has been some confusion in the literature regarding the terminology for counting the number of layers in such networks.

Thus, the network in Figure 6.9 is sometimes described as a three-layer network (which counts the number of layers of units and treats the inputs as units) or sometimes as a single-hidden-layer network (which counts the number of layers of hidden units).

We recommend a terminology in which Figure 6.9 is called a two-layer network, because it is the number of layers of learnable weights that is important for determining the network properties.

对于网络的层数，术语有所不同。本书建议将参数的层数称为网络的层数。

We have seen that a network of the form shown in Figure 6.9, having two layers of learnable parameters, has universal approximation capabilities.

However, networks with more than two layers can sometimes represent a given function with far fewer parameters than a two-layer network.

Montúfar et al. (2014) show that the network function divides the input space into a number of regions that is exponential in the depth of the network, but which is only polynomial in the width of the hidden layers.

神经网络的函数会将输入空间划分成许多区域，而这些区域的数量随着网络深度（层数）的增加呈指数级增长，但随着隐藏层宽度（每层单元数）的增加只呈多项式增长。

为了更清楚地理解，我们可以从以下几个方面来看：

1. 输入空间与区域划分：

输入空间是指所有可能的输入数据点构成的空间。例如，如果输入是一张 28×28 的灰度图像，那么每个像素的亮度值就构成了一个 784 维的输入空间。

神经网络（特别是使用 ReLU 等分段线性激活函数的网络）本质上是在这个高维空间中进行一系列的线性分割（由权重和偏置决定），从而将空间划分为不同的区域。在每个区域内，网络的行为（至少在理想情况下）可以近似为一个线性的函数组合。

2. 指数级依赖于深度 (Exponential in Depth):

“深度”指的是网络的层数。

这句话指出，如果你不断增加网络的层数（加深网络），那么理论上网络能够划分出的“线性区域”的数量会以指数形式（例如 2^L ，其中 L 是层数）急剧增加。

这意味着更深的网络有能力表示更复杂、更精细的决策边界和函数映射，因为它能将输入空间分割得更细碎、更复杂。

3. 多项式依赖于宽度 (Polynomial in Width):

“宽度”通常指网络中各隐藏层的神经元（或单元）数量。

这句话指出，如果你只是增加每一层的神经元数量（加宽网络），而不增加层数，那么理论上能划分出的“线性区域”的数量只会以多项式的形式（例如 W^d ，其中 W 是宽度， d 是某个常数）增长。

相比之下，这种增长速度远慢于指数增长。

这句话强调了网络深度对于提升模型表达能力的重要性。

仅仅增加每层的神经元数量（变宽），其对增加函数复杂度（即划分更多区域的能力）的贡献是相对有限的（多项式级别）。

而增加网络层数（变深），则能极大地提升网络划分输入空间的能力，使其能够学习和表示更加复杂和非线性的模式（指数级别）。

这为为什么深度学习（Deep Learning）有效提供了理论上的支持之一。

To represent the same function using a two-layer network would require an exponential number of hidden units.

如果要用一个只有两层（输入层到隐藏层，隐藏层到输出层）的浅层神经网络来表示同一个由深层网络

所能表达的复杂函数，那么这个两层网络需要拥有指数级增长的隐藏单元（神经元）数量才能做到。

深层网络：可以用较少的总参数（相对少的层数和宽度）来表示一个复杂的函数。

两层网络：如果想表示同样的复杂函数，则必须大大增加其隐藏层的宽度，其所需的神经元数量会随着问题复杂度（或输入维度）的增长而呈指数爆炸式增长。

这进一步突显了使用深度网络的优势：它们通过层次化结构，可以用更少的资源（更少的总神经元或参数）来捕捉和表示复杂的函数关系。

1 6.3.1 Hierarchical representations

Although this is an interesting result, a more compelling reason to explore deep neural networks is that the network architecture encodes a particular form of inductive bias, namely that the outputs are related to the input space through a hierarchical representation.

探索深度神经网络（Deep Neural Networks）的一个比之前提到的“用更少参数表示函数”的结果更令人信服的理由是，其网络结构本身蕴含了一种特定的归纳偏置（Inductive Bias）。

具体来说，这种归纳偏置指的是：网络假设（或倾向于学习）其输出是通过一种“层次化的表示”（Hierarchical Representation）与输入空间相关联的。

换句话说，深度网络的多层结构天然地适合学习从低级特征到高级概念的逐层抽象表示，而不是简单地用一个复杂的函数直接拟合输入和输出的关系。例如，在图像识别中，它会自动学习底层的边缘特征，再到中层的眼、耳等部件，最后到高层的“猫”这一概念。这种层次化处理方式是深度网络强大能力的关键原因之一。

A good example is the task of recognizing objects in images.

The relationship between the pixels of an image and a high-level concept such as ‘cat’ is highly complex and nonlinear, and would be an extremely challenging problem for a two-layer network.

However, a deep neural network can learn to detect low-level features, such as edges, in the early layers, and can then combine these in subsequent layers to make higher-level features such as eyes or whiskers, which in turn can be combined in later layers to detect the presence of a cat.

This can be viewed as a *compositional* inductive bias, in which higher-level objects, such as a cat, are composed of lower-level objects, such as eyes, which in turn have yet lower-level elements such as edges.

We can also think of this in reverse by considering the process of generating an image starting with low-level features such as edges, then combining these to form simple shapes such as circles, and then combining those in turn to form higher-level objects such as cats.

At each stage there are many ways to combine different components, giving an exponential gain in the number of possibilities with increasing depth.

这段话以图像识别为例，解释了深度网络的组合式归纳偏置：

问题：从像素直接识别“猫”这样的高层概念，关系高度复杂非线性，浅层网络很难做到。

深度网络的层级化学习：

浅层：学习低级特征，如边缘

中层：组合边缘形成更高层特征，如眼睛、胡须

深层：组合这些部件进一步检测出猫

组合性：高层物体（猫）由低层物体（眼睛）组成，低层物体又由更基础的边缘组成。

反向思考（生成过程）：从边缘开始 → 组合成简单形状（圆）→ 组合成高层物体（猫）。

指数级增益：每一层都有多种组合方式，深度增加使得可能的表示数量呈指数增长。这就是为什么深度网络比浅层网络强大得多。

2 6.3.2 Distributed representations

Neural networks can take advantage of another form of compositionality called a *distributed representation*.

Conceptually, each unit in a hidden layer can be thought of as representing a ‘feature’ at that level of the network, with a high value of the activation indicating that the corresponding feature is present and a low value indicating its absence.

With M units in a given layer, such a network can represent M different features.

However, the network could potentially learn a different representation in which *combinations* of hidden

units represent features, thereby potentially allowing a hidden layer with M units to represent 2^M different features, growing exponentially with the number of units.

Consider, for example, a network designed to process images of faces.

Each particular face image may or may not have glasses, it may or may not have a hat, it may or may not have a beard, leading to eight different combinations.

Although this could be represented by eight units each of which ‘turns on’ when it detects the corresponding combination, it could also be represented more compactly by just three units, one for each attribute.

These can be present independently of each other (although statistically their presence is likely to be correlated to some degree).

Later, we will explore in detail the kinds of internal representations that deep learning networks discover for themselves during training.

这段话解释了神经网络中的分布式表示概念：

基本想法：传统上，一个隐藏单元代表一个“特征”（激活值高表示特征存在，低表示不存在）。这样 M 个单元只能表示 M 种特征。

分布式表示的优势：网络可以学习用隐藏单元的组合来表示特征。这样 M 个单元理论上可以表示 2^M 种不同的特征组合，呈指数级增长。

例子：人脸图像可能有“是否戴眼镜”、“是否戴帽子”、“是否有胡子”三种属性。传统方式需要 $2^3 = 8$ 个单元（每个组合一个），但分布式表示只需 3 个单元（每个属性一个），这些属性可以独立组合。

核心要点：分布式表示通过组合方式大幅提升了网络的表示能力，这也是深度学习强大的重要原因之一。

3 6.3.3 Representation learning

We can view the successive layers of a deep neural network as performing transformations of the data, that make it easier to solve the desired task or tasks.

For example, a neural network that successfully learns to classify skin lesions as benign or malignant

must have learned to transform the original image data into a new space, represented by the outputs of the final layer of hidden units, such that the final layer of the network can distinguish the two classes.

This final layer can be viewed as a simple linear classifier, and so in the representation of the last hidden layer, the two classes must be well separated by a linear surface.

This ability to discover a nonlinear transformation of the data that makes subsequent tasks easier to solve is called *representation learning* (Bengio, Courville, and Vincent, 2012).

The learned representation, sometimes called the *embedding space*, is given by the outputs of one of the hidden layers of the network, so that any input vector, either from the training set or from some new data set, can be transformed into this representation by forward propagation through the network.

这段话解释了神经网络作为表示学习工具的核心思想：

逐层变换：网络的每一层都在对数据进行非线性变换，将原始数据逐步转换成更容易解决目标任务的形式。

一个例子：一个分类皮肤病变（良性/恶性）的网络，其最后一个隐藏层的输出构成了一个新的表示空间。在这一层中，两类样本可以被一个简单的线性分类器（即网络的最后一层）分开。

表示学习：网络自动学习这种能简化后续任务（如分类）的非线性变换的能力，就称为表示学习。

嵌入空间：这个学习到的表示（通常是某个隐藏层的输出）也叫嵌入空间。任何输入（训练集或新数据）都可以通过网络的前向传播映射到这个空间。

Representation learning is especially powerful because it allows us to exploit unlabelled data.

Often it is easy to collect a large quantity of unlabelled data, but acquiring the associated labels may be more difficult.

For example, a video camera on a vehicle can gather large numbers of images of urban scenes as the vehicle is driven around a city, but taking those images and identifying relevant objects, such as pedestrians and road signs, would require expensive and time-consuming human labelling.

这段话强调了表示学习在利用无标签数据方面的优势：

核心优势：表示学习能够利用大量无标签数据来学习有用的特征表示。

现实情况：收集大量无标签数据通常很容易（如车载摄像头拍摄的城市街景），但为这些数据打标签（如

标出行人、路标）则需要昂贵且耗时的人工标注。

应用价值：表示学习可以先从易得的无标签数据中学习通用特征，然后再用少量有标签数据进行微调，从而降低对大规模人工标注的依赖。

Learning from unlabelled data is called *unsupervised learning*, and many different algorithms have been developed to do this.

For example, a neural network can be trained to take images as input and to create the same images as the output.

To make this into a non-trivial task, the network may use hidden layers with fewer units than the number of pixels in the image, thereby forcing the network to learn some kind of compression of the images.

Only unlabelled data is needed because each image in the training set acts as both the input vector and the target vector.

Such networks are known as *autoencoders*.

The goal is that this type of training will force the network to discover some internal representation for the data that is useful for solving other tasks, such as image classification.

这段话介绍了无监督学习的一种典型方法 – 自编码器：

定义：从无标签数据中学习，称为无监督学习。

自编码器的工作原理：

网络以图像为输入，输出同一图像（即输入 = 目标）。

隐藏层的单元数小于图像的像素数，迫使网络学习对图像进行压缩。

数据需求：只需要无标签数据，因为每个图像既是输入也是目标。

目的：通过这种压缩训练，让网络发现数据中有用的内部表示，可用于其他任务（如图像分类）。

Historically, unsupervised learning played an important role in enabling the first deep networks (apart from convolutional networks) to be successfully trained.

Each layer of the network was first pre-trained using unsupervised learning and then the entire network was trained further using gradient-based supervised training.

It was later discovered that the pre-training phase could be omitted and a deep network could be trained from scratch purely using supervised learning given appropriate conditions.

However, pre-training and representation learning remain central to deep learning in other contexts.

The most notable example of pre-training is in natural language processing in which transformer models are trained on large quantities of text and are able to learn highly sophisticated internal representations of language that facilitates an impressive range of capabilities at human level and beyond.

这段话介绍了无监督预训练在深度学习发展中的作用及其现状：

历史作用：在早期（除卷积网络外），无监督学习对训练深层网络至关重要——先对每一层进行无监督预训练，再用监督学习对整个网络进行微调。

后来的发现：在合适的条件下（如足够的数据、合适的初始化等），可以跳过预训练阶段，直接从头用纯监督学习训练深层网络。

当前的地位：预训练和表示学习在其他领域仍然是核心方法。最典型的例子是自然语言处理中的 Transformer 模型 – 在大规模文本上预训练后，能学到非常精妙的语言内部表示，从而展现出超越人类的广泛能力。

4 6.3.4 Transfer learning

The internal representation learned for one particular task might also be useful for related tasks.

For example, a network trained on a large labelled data set of everyday objects can learn how to transform an image representation into one that is much better suited for classifying objects.

Then, the final classification layer of the network can be retrained using a smaller labelled data set of skin lesion images to create a lesion classifier.

This is an example of *transfer learning* (Hospedales et al., 2021), which allows higher accuracy to be achieved than if only the lesion image data were used for training, because the network can exploit commonalities shared by natural images in general.

Transfer learning is illustrated in Figure 6.13.

迁移学习：网络的早期层从第一个任务（如识别猫）中复制，然后对后几层在第二个任务（如识别病变）中重新训练。

In general, transfer learning can be used to improve performance on some task A, for which training data is in short supply, by using data from a related task B, for which data is more plentiful.

The two tasks should have the same kind of inputs, and there should be some commonality between the tasks so that low-level features, or internal representations, learned from task B will be useful for task A.

When we look at convolutional networks we will see that many image processing tasks require similar low-level features corresponding to the early layers of a deep neural network, whereas later layers are more specialized to a particular task, making such networks well suited to transfer learning applications.

这段话解释了迁移学习的基本思想及其在卷积网络中的适用性：

核心理念：当任务 A 的训练数据很少时，可以利用数据更丰富的任务 B 来提升任务 A 的性能。

前提条件：两个任务的输入类型相同，并且存在一定的共通性，这样从任务 B 学到的底层特征或内部表示对任务 A 也有用。

在卷积网络中的体现：

浅层/早期层：学习的是通用的底层特征（如边缘、颜色、纹理），对许多图像任务都适用。

深层/后期层：更专属于特定任务。

因此：可以先在大规模数据（如图像分类）上预训练，然后保留浅层、微调或替换深层来适应新任务——这正是迁移学习的典型做法。

When data for task A is very scarce, we might simply re-train the final layer of the network.

In contrast, if there are more data points, it is feasible to retrain several layers.

The process of learning parameters using one task that are then applied to one or more other tasks is called *pre-training*.

Note that for the new task, instead of applying stochastic gradient descent to the whole network, it is

much more efficient to send the new training data once through the fixed pre-trained network so as to evaluate the training inputs in the new representation.

Iterative gradient-based optimization can then be applied just to the smaller network consisting of the final layers.

As well as using a pre-trained network as a fixed pre-processor for a different task, it is also possible to apply *fine-tuning* in which the whole network is adapted to the data for task A.

This is generally done with a very small learning rate for a limited number of iterations to ensure that the network does not over-fit to the relatively small data set available for the new task.

这段话介绍了迁移学习中两种常见策略：冻结预训练网络进行特征提取，以及微调（fine-tuning）。

数据极少时（特征提取法）：只重新训练网络的最后一层（分类层），其余部分固定不动。

数据较多时：可以重新训练网络的多个层。

预训练：指先用一个任务（数据充足）学习参数，再将这些参数用于其他任务的过程。

高效做法：对于新任务，可以先用固定好的预训练网络一次性处理所有新数据，提取出新表示，然后再拿着这些表示只训练最后几层。这比每次都更新整个网络要快得多。

微调：在上述基础上，如果还想提升效果，可以用一个非常小的学习率对整个网络进行少量迭代训练，以防在较小的新数据集上过拟合。

5 6.3.5 Contrastive learning

One of the most common and powerful representation learning methods is *contrastive learning* (Gutmann and Hyvärinen, 2010; Oord, Li, and Vinyals, 2018; Chen, Kornblith, and others, 2020).

The idea is to learn a representation such that certain pairs of inputs, referred to as positive pairs, are close in the embedding space, and other pairs of inputs, called negative pairs, are far apart.

The intuition is that if we choose our positive pairs in such a way that they are semantically similar and choose negative pairs that are semantically dissimilar, then we will learn a representation space in which similar inputs are close, making downstream tasks, such as classification, much easier.

As with other forms of representation learning, the outputs of the trained network are typically not used directly, and instead the activations at some earlier layer are used to form the embedding space.

Contrastive learning is unlike most other machine learning tasks, in that the error function for a given input is defined only with respect to other inputs, instead of with a per-input label or target output.

对比学习的核心思想：

目标：学习一种表示（embedding），让“正样本对”（语义相似的输入）在空间中距离更近，让“负样本对”（语义不相似的输入）距离更远。

效果：这样得到的表示空间里，相似的输入会被聚集在一起，便于下游任务（如分类）。

备注：通常不直接使用网络最终输出，而是使用中间层的激活值作为表示。

特点：对比学习的损失函数依赖于“输入与其他输入的关系”，而不是像传统任务那样给每个输入一个独立标签。

Suppose we have a given data point $\mathbf{x}$ called the *anchor*, for which we have specified another data point $\mathbf{x}^+$ that together with $\mathbf{x}$ makes up a positive pair.

We must also specify a set of data points $\{\mathbf{x}_1^-, \dots, \mathbf{x}_N^-\}$ each of which makes up a negative pair with $\mathbf{x}$.

We now need a loss function that will reward close proximity between the representations of $\mathbf{x}$ and $\mathbf{x}^+$ while encouraging a large distance between each pair $\{\mathbf{x}, \mathbf{x}_n^-\}$.

One example of such a function, and the most commonly used loss function for contrastive learning, is called the *InfoNCE* loss (Gutmann and Hyvärinen, 2010; Oord, Li, and Vinyals, 2018), where NCE denotes ‘noise contrastive estimation’.

对比学习中如何构造损失函数，特别是 InfoNCE 损失：

基本设定：有一个“锚点” $\mathbf{x}$ ，一个正样本 $\mathbf{x}^+$ ，和 N 个负样本 $\{\mathbf{x}_1^-, \dots, \mathbf{x}_N^-\}$ 。

损失函数的目标：让锚点与正样本的表示尽量接近，同时让锚点与每个负样本的表示尽量远离。

常用方法：InfoNCE 损失（NCE 代表 Noise Contrastive Estimation 噪声对比估计），是对比学习中最常用的损失函数之一。

Suppose we have a neural network function $\mathbf{f}_w(\mathbf{x})$ that maps points from the input space $\mathbf{x}$ to a repre-

sentation space, governed by learnable parameters $\mathbf{w}$.

This representation is normalized so that $\|\mathbf{f}_{\mathbf{w}}(\mathbf{x})\| = 1$.

InfoNCE 损失的公式及原理:

设置: 神经网络 $\mathbf{f}_{\mathbf{w}}(\mathbf{x})$ 将输入映射到归一化的表示空间 (长度恒为 1), 因此向量的点积等于余弦相似度。

Then, for a data point $\mathbf{x}$, the InfoNCE loss is defined by

$$E(\mathbf{w}) = -\ln \frac{\exp(\mathbf{f}_{\mathbf{w}}(\mathbf{x})^{\top} \mathbf{f}_{\mathbf{w}}(\mathbf{x}^+))}{\exp(\mathbf{f}_{\mathbf{w}}(\mathbf{x})^{\top} \mathbf{f}_{\mathbf{w}}(\mathbf{x}^+)) + \sum_{n=1}^N \exp(\mathbf{f}_{\mathbf{w}}(\mathbf{x})^{\top} \mathbf{f}_{\mathbf{w}}(\mathbf{x}_n^-))}. \quad (6.20)$$

损失函数 (公式 6.20):

分子: 锚点与正样本之间的相似度 $\exp(\text{相似度})$ 。

分母: 正样本相似度 + 所有负样本相似度的和。

整体取负对数 $\rightarrow$ 希望正样本的相似度远大于负样本。

We can see that in this function, the cosine similarity $\mathbf{f}_{\mathbf{w}}(\mathbf{x})^{\top} \mathbf{f}_{\mathbf{w}}(\mathbf{x}^+)$ between the representation $\mathbf{f}_{\mathbf{w}}(\mathbf{x})$ of the anchor and the representation $\mathbf{f}_{\mathbf{w}}(\mathbf{x}^+)$ of the positive example provides our measure of how close the positive pair examples are in the learned space, and the same measure is used to assess how close the anchor is to the negative examples.

Note that the function resembles a classification cross-entropy error function in which the cosine similarity of the positive pair gives the logit for the label class and the cosine similarities for the negative pairs give the logits for the incorrect classes.

直观理解: 这类似于分类问题中的交叉熵损失 – 把正样本当作 “正确类别”, 负样本当作 “错误类别”, 相似度作为 logits。

Also note that the negative pairs are crucial as without them the embedding would simply learn the degenerate solution of mapping every point to the same representation.

负样本的重要性: 如果没有负样本, 模型会直接学到把全部输入映射到同一个点 (平凡解), 从而失去区分能力。

A particular contrastive learning algorithm is defined predominantly by how the positive and negative pairs are chosen, which is how we use our prior knowledge to specify what a good representation should be.

For example, consider the problem of learning representations of images.

Here, a common choice is to create positive pairs by corrupting the input images in ways that should preserve the semantic information of the image while greatly altering the image in the pixel space (Wu et al., 2018; He et al., 2019; Chen, Kornblith, et al., 2020).

Corruptions are closely related to *data augmentations*, and examples include rotation, translation, and colour shifts.

Other images from the data set can then be used to create the negative pairs.

This approach to contrastive learning is known as *instance discrimination*.

这段话解释了对比学习算法的核心在于如何选择正负样本对，并给出了图像领域的一个具体例子 – 实例判别（instance discrimination）：

正样本对的构造：对同一张图像进行数据增强（如旋转、平移、颜色变化等），这些改变在像素层面很大，但保留语义信息。增强前后的图像组成正样本对。

负样本对的构造：使用数据集中的其他图像（通常是不同的原始图像）作为负样本。

核心思想：模型需要学会将同一图像的不同增强版本映射到相近的表示，而将不同图像的表示分开。这种方法被称为实例判别。

If, however, we have access to class labels, then we can use images of the same class as positive pairs and images of different classes as negative pairs.

This relaxes the reliance on specifying the augmentations that the representation should be invariant to and also avoids treating two semantically similar images as a negative pair.

This is referred to as supervised contrastive learning (Khosla et al, 2020) because of the reliance on the class labels, and it can often yield better results than simply learning the representation using cross-entropy classification.

这段话介绍了监督对比学习：

与无监督对比学习的区别：不再靠数据增强构造正样本对，而是利用类别标签 – 同一类别的图像作为正样本对，不同类别的图像作为负样本对。

优点：

不需要手动设计数据增强方式。

避免把语义相似的图像错误地当作负样本对。

效果：通常比直接用交叉熵分类学习表示的效果更好。

The members of positive and negative pairs do not necessarily have to come from the same data modality.

In contrastive-language image pretraining, or CLIP (Radford et al., 2021), a positive pair consists of an image and its corresponding text caption, and two separate functions, one for each modality, are used to map the inputs to the same representation space.

Negative pairs are then mismatched images and captions.

This is often referred to as *weakly supervised*, as it relies on captioned images, which are often easier to obtain by scraping data from the internet than by manually labelling images with their classes.

The loss function in this case is given by

$$E(\mathbf{w}) = -\frac{1}{2} \ln \frac{\exp(\mathbf{f}_{\mathbf{w}}(\mathbf{x}^+)^\top \mathbf{g}_{\mathbf{w}}(\mathbf{y}^+))}{\exp(\mathbf{f}_{\mathbf{w}}(\mathbf{x}^+)^\top \mathbf{g}_{\mathbf{w}}(\mathbf{y}^+)) + \sum_{n=1}^N \exp(\mathbf{f}_{\mathbf{w}}(\mathbf{x}_n^-)^\top \mathbf{g}_{\mathbf{w}}(\mathbf{y}^+))} - \frac{1}{2} \ln \frac{\exp(\mathbf{f}_{\mathbf{w}}(\mathbf{x}^+)^\top \mathbf{g}_{\mathbf{w}}(\mathbf{y}^+))}{\exp(\mathbf{f}_{\mathbf{w}}(\mathbf{x}^+)^\top \mathbf{g}_{\mathbf{w}}(\mathbf{y}^+)) + \sum_{m=1}^M \exp(\mathbf{f}_{\mathbf{w}}(\mathbf{x}^+)^\top \mathbf{g}_{\mathbf{w}}(\mathbf{y}_m^-))} \quad (6.21)$$

where $\mathbf{x}^+$ and $\mathbf{y}^+$ represent a positive pair in which $\mathbf{x}$ is an image and $\mathbf{y}$ is its corresponding text caption, $\mathbf{f}_{\mathbf{w}}$ represents the mapping from images to the representation space, and $\mathbf{g}_{\mathbf{w}}$ is the mapping from text input to the representation space.

We also require a set $\{\mathbf{x}_1^-, \dots, \mathbf{x}_N^-\}$ of other images from the data set, for which we assume the text caption $\mathbf{y}^+$ is inappropriate, and a set $\{\mathbf{y}_1^-, \dots, \mathbf{y}_M^-\}$ of text captions that are similarly mismatched to the input image $\mathbf{x}$.

The two terms in the loss function ensure that (a) the representation of the image is close to the text caption representation relative to other image representations and (b) the text caption representation is close to the representation of the image it describes relative to other representations of text captions.

Although CLIP uses text and image pairs, any data set with paired modalities can be used to learn representations.

A comparison of the different contrastive learning methods we have discussed is shown in Figure 6.14.

这段话介绍了多模态对比学习，以 CLIP 为例：

正负样本构造：正样本对由图像与其对应的文字描述组成，负样本对是不匹配的图像与文字（比如一张猫的图片配上“一只狗”的描述）。

两个编码器：图像编码器 $\mathbf{f}_w$ 和文本编码器 $\mathbf{g}_w$ ，将不同模态的数据映射到同一个表示空间。

损失函数（公式 6.21）包含两项：

1. 让图像与它匹配的文字描述更近，同时远离其他不匹配的图像。
2. 让文字描述与它匹配的图像更近，同时远离其他不匹配的文字描述。

特点：

属于弱监督学习，因为图像-文字对可以通过网络爬取获得，比手工标注类别标签更容易。

不限于图像和文字，任何成对的不同模态数据都可以用这个思路。

6 6.3.6 General network architectures

So far, we have explored neural network architectures that are organized into a sequence of fully-connected layers.

However, because there is a direct correspondence between a network diagram and its mathematical function, we can develop more general network mappings by considering more complex network diagrams.

These must be restricted to a *feed-forward* architecture, in other words to one having no closed directed cycles, to ensure that the outputs are deterministic functions of the inputs.

前馈架构：无有向闭环，保证函数定义确定。

This is illustrated with a simple example in Figure 6.15.

Each (hidden or output) unit in such a network computes a function given by

$$z_k = h \left(\sum_{j \in \mathcal{A}(k)} w_{kj} z_j + b_k \right) \quad (6.22)$$

where $\mathcal{A}(k)$ denotes the set of *ancestors* of node k , in other words the set of units that send connections to unit k , and b_k denotes the associated bias parameter.

For a given set of values applied to the inputs of the network, successive application of (6.22) allows the activations of all units in the network to be evaluated including those of the output units.

这段话介绍了更通用的前馈神经网络架构：

背景：之前讨论的是全连接层序列组成的网络，但网络图可以直接对应数学函数，因此可以设计更复杂的结构。

限制条件：必须满足前馈结构，即没有有向闭环（不能有循环连接），这样才能保证输出是输入的确定性函数。

计算方法：每个单元（隐藏层或输出层）的值按公式 (6.22) 计算，其中 $\mathcal{A}(k)$ 表示单元 k 的祖先节点集合（即连接到 k 的所有上游单元）。

执行顺序：给定输入后，按依赖关系逐层依次计算所有单元的激活值，直到得到输出。

7 6.3.7 Tensors

We see that linear algebra plays a central role in neural networks, with quantities such as data sets, activations, and network parameters represented as scalars, vectors, and matrices.

However, we also encounter variables of higher dimensionality.

Consider, for example, a data set of N colour images each of which is I pixels high and J pixels wide.

Each pixel is indexed by its row and column within the image and has red, green, and blue values.

We have one such value for each image in the data set, and so we can represent a particular intensity

value by a four-dimensional array $\mathbf{X}$ with elements x_{ijkn} , where $i \in \{1, \dots, I\}$ and $j \in \{1, \dots, J\}$ index the row and column within the image, $k \in \{1, 2, 3\}$ indexes the red, green, and blue intensities, and $n \in \{1, \dots, N\}$ indexes the particular image within the data set.

These higher-dimensional arrays are called *tensors* and include scalars, vectors, and matrices as special cases.

We will see many examples of such tensors when we discuss more sophisticated neural network architectures later in the book.

Massively parallel processors such as GPUs are especially well suited to processing tensors.